

Graphs: Basics and Traversals

A Problem-Solving Companion to Striver's A2Z and SDE Sheets

Maruri Saikiran

June 30, 2026

Contents

1	Graphs: Basics and Traversals	1
1.1	Representing a Graph, and the Two Ways to Walk It	1
1.2	Depth-First Search (DFS) Traversal	3
1.3	Breadth-First Search (BFS) Traversal	4
1.4	Number of Provinces	6
1.5	Clone a Graph	8
1.6	Rotten Oranges	10
1.7	Flood Fill	12
1.8	01 Matrix	14
1.9	Number of Islands	15

Chapter 1

Graphs: Basics and Traversals

A graph is nothing more than a collection of states and the moves that connect them. Once a problem's "states" and "valid moves" are identified, traversing the graph – visiting every reachable state exactly once – answers a surprising number of questions that don't look like graph problems at first glance.

1.1 Representing a Graph, and the Two Ways to Walk It

A graph is a set of **nodes** (vertices) and **edges** (connections between nodes, which may be directed or undirected, weighted or unweighted). The standard representation for sparse graphs (the vast majority of interview problems) is an **adjacency list**: an array (or hash map) where index i holds a list of every node directly reachable from node i .

Recognizing a Graph Problem

A problem is a graph problem in disguise whenever it describes **states** and **transitions between states** – grid cells and their neighbors, people and their connections, words and single-character edits, courses and their prerequisites. The moment a problem can be rephrased as “starting from X, what can I reach by repeatedly taking valid moves,” a graph traversal applies.

Depth-First Search vs. Breadth-First Search

Both traversals visit every node reachable from a start node exactly once, using a *visited* array to avoid repeats (and infinite loops in cyclic graphs) – they differ only in **which data structure** drives the order of exploration:

- **DFS** uses a **stack** (often the recursion call stack itself): go as deep as possible down one path before backtracking. Natural for problems about *connectivity* and *exhaustive exploration* (does a path exist, what is everything reachable).
- **BFS** uses an explicit **queue**: explore all nodes at distance 1 before any node at distance 2. Natural for problems about *shortest distance in an unweighted graph* or *level-by-level* processing (multi-source spreading, like fire or rot spreading outward).

What This Chapter Covers

- **DFS Traversal (Section 1.2)** and **BFS Traversal (Section 1.3)**: the two foundational walks, established once and reused by name in every later chapter.
- **Number of Provinces (Section 1.4)**: counting connected components – run a traversal from every unvisited node, counting how many traversals it takes.
- **Clone a Graph (Section 1.5)**: a traversal that builds a parallel copy of the graph

as it visits, using a hash map to avoid duplicating already-cloned nodes.

- **Rotten Oranges (Section 1.6), Flood Fill (Section 1.7), and 01 Matrix (Section 1.8):** treating a grid as an implicit graph (each cell a node, its 4 neighbors its edges), with the first and last introducing **multi-source BFS** – starting a single BFS from *every* source simultaneously.
- **Number of Islands (Section 1.9):** Number of Provinces’ connected-component counting, applied to an implicit grid graph instead of an explicit adjacency list.

1.2 Depth-First Search (DFS) Traversal

Problem Statement. Given a graph (as an adjacency list) and a starting node, visit every node reachable from the start exactly once, exploring as deep as possible along each path before backtracking.

Pattern Recognition

“Visit every reachable node, going deep before wide” is solved by recursion (or an explicit stack): visit the current node, mark it visited, then recurse into every unvisited neighbor in turn.

Algorithm

1. Maintain a *visited* array, initialized to all false.
2. Define *dfs(node)*: mark *node* as visited, add it to the result; for every neighbor of *node* not yet visited, recursively call *dfs(neighbor)*.
3. To traverse the whole graph (which may be disconnected), call *dfs(i)* for every node *i* not yet visited.

Why a *visited* array is required

Without tracking visited nodes, a cycle in the graph (or even a simple back-and-forth edge between two nodes) would cause infinite recursion. Marking a node visited **before** recursing into its neighbors (not after returning) is essential: it guarantees that if two different neighbors both point back to an already-in-progress node, only the first one to reach it recurses into it.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void dfs(int node, vector<vector<int>>& adj, vector<bool>& visited,
5         vector<int>& result) {
6     visited[node] = true;
7     result.push_back(node);
8
9     for (int neighbor : adj[node]) {
10         if (!visited[neighbor]) {
11             dfs(neighbor, adj, visited, result);
12         }
13     }
14 }
15
16 vector<int> dfsTraversal(int n, vector<vector<int>>& adj) {
17     vector<bool> visited(n, false);
18     vector<int> result;
19
20     for (int i = 0; i < n; i++) {
21         if (!visited[i]) {
22             dfs(i, adj, visited, result);
23         }
24     }
25     return result;
26 }
```

```

27
28 int main() {
29     int n = 5;
30     vector<vector<int>> adj(n);
31     auto addEdge = [&](int u, int v) { adj[u].push_back(v); adj[v].push_back(u); };
32     addEdge(0, 1); addEdge(0, 2); addEdge(1, 3); addEdge(2, 4);
33
34     for (int x : dfsTraversal(n, adj)) cout << x << " ";
35     cout << endl; // 0 1 3 2 4
36     return 0;
37 }

```

Dry run

Setup. Take the graph with edges 0–1, 0–2, 1–3, 2–4, starting traversal from node 0.

Visit 0 (mark visited, record). Its neighbors: 1 (unvisited) – recurse. Visit 1, record. Its neighbors: 0 (visited, skip), 3 (unvisited) – recurse. Visit 3, record; no unvisited neighbors, return. Back at 1: no more neighbors, return. Back at 0: next neighbor 2 (unvisited) – recurse. Visit 2, record. Its neighbors: 0 (visited, skip), 4 (unvisited) – recurse. Visit 4, record; no unvisited neighbors, return. Final order: 0, 1, 3, 2, 4.

Complexity

Time Complexity. $O(V + E)$ – every vertex is visited once (marking it visited), and every edge is examined once (from each endpoint, checking if the other side is visited).

Space Complexity. $O(V)$ for the *visited* array and the recursion stack (which can be as deep as V in a path-shaped graph).

Takeaway

DFS’s “visit, mark, recurse into unvisited neighbors” template is the single most reused piece of code in this chapter – nearly every later problem either calls this function directly or modifies one small part of it (the visiting action, the stopping condition, or what gets collected).

1.3 Breadth-First Search (BFS) Traversal

Problem Statement. Given a graph (as an adjacency list) and a starting node, visit every node reachable from the start exactly once, exploring all nodes at distance 1 before any node at distance 2, and so on.

Pattern Recognition

“Visit level by level, nearest first” is solved with a **queue**: push the start node, then repeatedly pop the front, visit it, and push every unvisited neighbor onto the back – the first-in-first-out order of a queue naturally processes nodes in increasing distance from the start.

Algorithm

1. Maintain a *visited* array; mark the start node visited and push it onto a queue.

2. While the queue is non-empty: pop the front node, add it to the result; for every unvisited neighbor, mark it visited **immediately** (before it is popped) and push it onto the queue.
3. To traverse the whole graph, repeat for every node not yet visited.

Mark Visited on Push, Not on Pop

A subtle but important detail: a node must be marked visited the moment it is **pushed** onto the queue, not when it is popped. If marking were delayed until pop time, the same node could be pushed multiple times by different neighbors before any of those pushes is processed, wasting work and (in graphs with cycles) potentially causing incorrect results.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> bfsFromSource(int src, vector<vector<int>>& adj, vector<bool>& visited)
5  {
6      vector<int> result;
7      queue<int> q;
8      visited[src] = true;
9      q.push(src);
10
11     while (!q.empty()) {
12         int node = q.front();
13         q.pop();
14         result.push_back(node);
15
16         for (int neighbor : adj[node]) {
17             if (!visited[neighbor]) {
18                 visited[neighbor] = true;
19                 q.push(neighbor);
20             }
21         }
22     }
23     return result;
24 }
25
26 vector<int> bfsTraversal(int n, vector<vector<int>>& adj) {
27     vector<bool> visited(n, false);
28     vector<int> result;
29
30     for (int i = 0; i < n; i++) {
31         if (!visited[i]) {
32             vector<int> part = bfsFromSource(i, adj, visited);
33             result.insert(result.end(), part.begin(), part.end());
34         }
35     }
36     return result;
37 }
38
39 int main() {
40     int n = 5;
41     vector<vector<int>> adj(n);
42     auto addEdge = [&](int u, int v) { adj[u].push_back(v); adj[v].push_back(u); };
43     addEdge(0, 1); addEdge(0, 2); addEdge(1, 3); addEdge(2, 4);
44
45     for (int x : bfsTraversal(n, adj)) cout << x << " ";

```

```

45     cout << endl; // 0 1 2 3 4
46     return 0;
47 }

```

Dry run

Setup. Take the same graph as Section 1.2: edges 0–1, 0–2, 1–3, 2–4, starting from node 0.

Queue: [0] (visited = {0}). Pop 0, record; push unvisited neighbors 1, 2: queue [1, 2] (visited = {0, 1, 2}). Pop 1, record; push unvisited neighbor 3: queue [2, 3]. Pop 2, record; push unvisited neighbor 4: queue [3, 4]. Pop 3, record; no new neighbors. Pop 4, record; no new neighbors. Final order: 0, 1, 2, 3, 4 – notice this differs from DFS’s 0, 1, 3, 2, 4: BFS visits both of 0’s direct neighbors (1 and 2) before descending to either of their children, exactly the “nearest first” property.

Complexity

Time Complexity. $O(V + E)$ – identical bound to DFS, for the same reason (every vertex and edge examined once). **Space Complexity.** $O(V)$ for the *visited* array and the queue (which can hold up to all V nodes in the worst case, e.g. a star graph).

Takeaway

DFS and BFS visit the exact same set of reachable nodes and share the same $O(V + E)$ complexity – the only difference is traversal **order**, governed entirely by swapping a stack for a queue. Whenever a problem specifically needs shortest paths in an unweighted graph or strictly level-by-level processing, that need points directly at BFS; everything else defaults naturally to whichever is more convenient to implement (usually DFS, via plain recursion).

1.4 Number of Provinces

Problem Statement. Given an $n \times n$ adjacency matrix *isConnected* for n cities (where *isConnected*[i][j] = 1 means cities i and j are directly connected), find the number of **provinces** – groups of cities connected directly or indirectly, where a city in one group has no connection to a city in any other group.

Pattern Recognition

“Count the number of disconnected groups” is exactly counting **connected components**: run a traversal (DFS or BFS) from every **unvisited** node, marking everything that traversal reaches. Each time a traversal is launched from a fresh, unvisited node, that is one new province discovered.

Algorithm

1. Maintain a *visited* array and a *provinces* counter, both initialized to 0/false.
2. For each city i from 0 to $n - 1$: if i is not visited, increment *provinces*, then run a full DFS (or BFS) from i , marking every reachable city visited.
3. Return *provinces*.

Why This Always Terminates with the Correct Count

Every city belongs to exactly one province (provinces are, by definition, a partition of the cities). The traversal from an unvisited city visits *exactly* that city's entire province (no more, no less, since the adjacency matrix's connections define the province's boundary). So each outer-loop iteration that finds an unvisited city is necessarily the *first* city of a province not yet counted – which is precisely why incrementing the counter there, and nowhere else, gives the correct total.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void dfs(int node, vector<vector<int>>& isConnected, vector<bool>& visited) {
5      visited[node] = true;
6      int n = isConnected.size();
7      for (int neighbor = 0; neighbor < n; neighbor++) {
8          if (isConnected[node][neighbor] == 1 && !visited[neighbor]) {
9              dfs(neighbor, isConnected, visited);
10         }
11     }
12 }
13
14 int findCircleNum(vector<vector<int>>& isConnected) {
15     int n = isConnected.size();
16     vector<bool> visited(n, false);
17     int provinces = 0;
18
19     for (int i = 0; i < n; i++) {
20         if (!visited[i]) {
21             provinces++;
22             dfs(i, isConnected, visited);
23         }
24     }
25     return provinces;
26 }
27
28 int main() {
29     vector<vector<int>> isConnected = {
30         {1, 1, 0},
31         {1, 1, 0},
32         {0, 0, 1}
33     };
34     cout << findCircleNum(isConnected) << endl; // 2
35     return 0;
36 }

```

Dry run

Setup. Take the 3×3 matrix above (cities 0, 1 connected; city 2 isolated).

$i = 0$: unvisited, *provinces* = 1; DFS from 0 visits 1 (connected) and stops (city 2 not connected to 0 or 1); visited = {0, 1}. $i = 1$: already visited, skip. $i = 2$: unvisited, *provinces* = 2; DFS from 2 visits only itself. Final answer: 2 provinces.

Complexity

Time Complexity. $O(n^2)$ – the adjacency matrix representation forces an $O(n)$ scan per node during DFS (checking every possible neighbor), across n nodes total. **Space Complexity.** $O(n)$ for the *visited* array and recursion stack.

Takeaway

“Count the number of connected components” is one of the most frequently recurring graph templates: launch a traversal from every unvisited node, counting launches. This exact loop structure – *not* the traversal itself – is what changes across this chapter’s later problems (Number of Islands, Section 1.9, reuses this identical loop on an implicit grid graph instead of an explicit matrix).

1.5 Clone a Graph

Problem Statement. Given a reference to a node in a connected undirected graph (where each node stores a value and a list of neighbor pointers), return a **deep copy** (clone) of the entire graph – a structurally identical graph made of entirely new node objects.

Pattern Recognition

“Copy every node and every connection, visiting each node exactly once” is a traversal (DFS or BFS) augmented with a **hash map from original node to its clone** – the map serves double duty: it remembers which nodes have already been cloned (acting as the traversal’s *visited* check) and it provides the correct clone object to link into a neighbor’s adjacency list.

Why a hash map, not a simple *visited* array

Every traversal in this chapter needs to track which nodes have been visited – but previous problems used array indices (city numbers, grid coordinates) as keys, which a simple boolean array handles directly. Here, nodes are objects with no natural array index, and *visiting* a node is inseparable from *creating its clone* – so a hash map from original node pointer to cloned node pointer plays both roles: checking `map.find(node) ≠ map.end()` is the visited check, and `map[node]` retrieves the already-built clone to attach as a neighbor.

Algorithm (DFS)

1. Define `cloneNode(node)`: if `node` is already in the map, return its clone immediately (this breaks cycles – without this check, a cyclic graph would recurse forever).
2. Otherwise, create a new node with the same value, store it in the map **before** recursing into neighbors (exactly like marking visited before recursing in plain DFS), then for each original neighbor, recursively clone it and append the result to the new node’s neighbor list.
3. Return `cloneNode(startNode)`.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct Node {
5      int val;
6      vector<Node*> neighbors;
7      Node(int v) : val(v) {}
8  };
9
10 Node* cloneNode(Node* node, unordered_map<Node*, Node*>& cloneMap) {
11     if (cloneMap.find(node) != cloneMap.end()) {
12         return cloneMap[node];
13     }
14
15     Node* clone = new Node(node->val);
16     cloneMap[node] = clone;
17
18     for (Node* neighbor : node->neighbors) {
19         clone->neighbors.push_back(cloneNode(neighbor, cloneMap));
20     }
21     return clone;
22 }
23
24 Node* cloneGraph(Node* startNode) {
25     if (startNode == nullptr) return nullptr;
26     unordered_map<Node*, Node*> cloneMap;
27     return cloneNode(startNode, cloneMap);
28 }
29
30 int main() {
31     Node* a = new Node(1);
32     Node* b = new Node(2);
33     Node* c = new Node(3);
34     a->neighbors = {b, c};
35     b->neighbors = {a, c};
36     c->neighbors = {a, b};
37
38     Node* clonedA = cloneGraph(a);
39     cout << clonedA->val << " has " << clonedA->neighbors.size()
40          << " neighbors" << endl; // 1 has 2 neighbors
41     cout << (clonedA == a ? "same object" : "different object") << endl; //
42          different object
43     return 0;
44 }

```

Dry run

Setup. Take a triangle graph: nodes 1,2,3, each connected to the other two. Start cloning from node 1.

cloneNode(1): not in map, create clone 1', store $map[1] = 1'$. Recurse into neighbor 2: *cloneNode(2)*: not in map, create clone 2', store $map[2] = 2'$. Recurse into 2's neighbors, 1 and 3. *cloneNode(1)*: **already in map**, return 1' immediately (no infinite recursion, even though 1 and 2 point back to each other). *cloneNode(3)*: not in map, create clone 3', store $map[3] = 3'$; recurse into 3's neighbors 1,2, both already in map, return 1',2' immediately. Back at 2': neighbors = [1',3']. Back at 1': continue to neighbor 3, already in map, return 3'. Final: 1's neighbors = [2',3'] – a complete, independent clone of the original triangle.

Complexity

Time Complexity. $O(V + E)$ – every node is cloned exactly once (guarded by the map check), and every edge is examined exactly once (from each direction). **Space Complexity.** $O(V)$ for the hash map and the recursion stack.

Takeaway

Whenever a traversal needs to **build something new** while visiting (a clone, a transformed copy, a mapping to a different representation) rather than simply recording or counting, a hash map keyed by the original node typically replaces the plain *visited* array – the map’s presence-check still prevents revisiting and infinite loops, while also handing back the already-built result the moment it’s needed again.

1.6 Rotten Oranges

Problem Statement. Given a grid where each cell is 0 (empty), 1 (fresh orange), or 2 (rotten orange), every minute any fresh orange **4-directionally adjacent** to a rotten orange becomes rotten. Find the minimum number of minutes until no fresh orange remains, or -1 if some fresh orange can never be reached.

Pattern Recognition

“Multiple sources spread outward simultaneously, find the time for the spread to finish” is solved by **multi-source BFS**: push **every** initially rotten orange into the queue at the start (not just one), then run ordinary BFS – the key realization is that BFS already processes nodes in order of distance from *whichever* source is nearest, automatically, when all sources start in the queue together at time 0.

Why starting from all sources at once is correct

If BFS were run separately from each rotten orange one at a time, the correctness would still hold but the bookkeeping (combining several separate distance computations, taking a minimum over all of them) would be needlessly complex. Pushing every source into the *same* queue at the very start works because BFS’s queue already guarantees nodes are processed in non-decreasing distance order *from whichever source reaches them first* – exactly the quantity this problem wants (a fresh orange rots at the minute the *nearest* rotten orange’s spread reaches it).

Algorithm

1. Scan the grid: push every rotten orange’s coordinates into a queue, and count the total number of fresh oranges.
2. Run BFS level by level (processing the entire current queue before moving to the next level represents one minute passing): for each rotten orange popped, check its 4 neighbors; any fresh neighbor becomes rotten, decrement the fresh count, and push it for the next level.
3. Track the number of levels (minutes) processed. After BFS completes, if any fresh oranges remain uncounted, return -1 ; otherwise return the number of minutes elapsed.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int orangesRotting(vector<vector<int>>& grid) {
5     int rows = grid.size(), cols = grid[0].size();
6     queue<pair<int,int>> q;
7     int freshCount = 0;
8
9     for (int r = 0; r < rows; r++) {
10         for (int c = 0; c < cols; c++) {
11             if (grid[r][c] == 2) q.push({r, c});
12             else if (grid[r][c] == 1) freshCount++;
13         }
14     }
15
16     int minutes = 0;
17     int dr[] = {-1, 1, 0, 0};
18     int dc[] = {0, 0, -1, 1};
19
20     while (!q.empty() && freshCount > 0) {
21         int levelSize = q.size();
22         bool rottedThisLevel = false;
23
24         for (int i = 0; i < levelSize; i++) {
25             auto [r, c] = q.front();
26             q.pop();
27
28             for (int dir = 0; dir < 4; dir++) {
29                 int nr = r + dr[dir], nc = c + dc[dir];
30                 if (nr >= 0 && nr < rows && nc >= 0 && nc < cols &&
31                     grid[nr][nc] == 1) {
32                     grid[nr][nc] = 2;
33                     freshCount--;
34                     q.push({nr, nc});
35                     rottedThisLevel = true;
36                 }
37             }
38         }
39         if (rottedThisLevel) minutes++;
40     }
41     return freshCount == 0 ? minutes : -1;
42 }
43
44 int main() {
45     vector<vector<int>> grid = {
46         {2, 1, 1},
47         {1, 1, 0},
48         {0, 1, 1}
49     };
50     cout << orangesRotting(grid) << endl; // 4
51     return 0;
52 }

```

Dry run

Setup. Take the 3×3 grid above (one rotten orange at $(0,0)$).

Initial queue: $[(0,0)]$, $freshCount = 6$. Minute 1: pop $(0,0)$, rot neighbors $(1,0)$ and $(0,1)$ (both fresh); push both; $freshCount = 4$. Minute 2: pop $(1,0)$, rot $(1,1)$; pop $(0,1)$, rot $(0,2)$ ($(1,1)$ is already rotten by this point, so $(0,1)$'s other neighbor is skipped); $freshCount = 2$.

Minute 3: pop (1, 1), rot (2, 1); pop (0, 2), no fresh neighbors; *freshCount* = 1. Minute 4: pop (2, 1), rot (2, 2); *freshCount* = 0. The queue still holds (2, 2), but the loop exits immediately since *freshCount* = 0 – final answer: 4 minutes.

Complexity

Time Complexity. $O(\text{rows} \times \text{cols})$ – every cell is pushed onto the queue and processed at most once. **Space Complexity.** $O(\text{rows} \times \text{cols})$ for the queue in the worst case (a grid that is entirely rotten oranges initially).

Takeaway

Multi-source BFS – seeding the queue with every source simultaneously rather than running BFS once per source – is the standard technique whenever a problem describes several things spreading or growing outward at the same rate and asks for the time until the spread finishes or the distance to the nearest source. The very next section, 01 Matrix, applies this exact same technique to a differently-phrased problem.

1.7 Flood Fill

Problem Statement. Given a 2D grid of pixel colors, a starting pixel (*sr*, *sc*), and a *newColor*, change the color of the starting pixel and every pixel **4-directionally connected** to it through pixels of the **same original color** to *newColor*.

Pattern Recognition

“Change every connected pixel of the same color” is a direct single-source DFS (or BFS) on the implicit grid graph, where two pixels are connected exactly when they are 4-directional neighbors **and** share the starting pixel’s original color.

Algorithm

1. Record *originalColor* = *image*[*sr*][*sc*].
2. If *originalColor* = *newColor*, return the image unchanged immediately (otherwise the DFS below would recurse infinitely, since every newly-colored pixel would still match the “original color” check).
3. DFS from (*sr*, *sc*): if the current pixel is in bounds and equals *originalColor*, set it to *newColor*, then recurse into all 4 neighbors.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void fill(vector<vector<int>>& image, int r, int c, int originalColor, int newColor
5 ) {
6     int rows = image.size(), cols = image[0].size();
7     if (r < 0 || r >= rows || c < 0 || c >= cols) return;
8     if (image[r][c] != originalColor) return;
9     image[r][c] = newColor;
```

```

10     fill(image, r - 1, c, originalColor, newColor);
11     fill(image, r + 1, c, originalColor, newColor);
12     fill(image, r, c - 1, originalColor, newColor);
13     fill(image, r, c + 1, originalColor, newColor);
14 }
15
16 vector<vector<int>> floodFill(vector<vector<int>>& image, int sr, int sc, int
    newColor) {
17     int originalColor = image[sr][sc];
18     if (originalColor != newColor) {
19         fill(image, sr, sc, originalColor, newColor);
20     }
21     return image;
22 }
23
24 int main() {
25     vector<vector<int>> image = {
26         {1, 1, 1},
27         {1, 1, 0},
28         {1, 0, 1}
29     };
30     vector<vector<int>> result = floodFill(image, 1, 1, 2);
31     for (auto& row : result) {
32         for (int x : row) cout << x << " ";
33         cout << endl;
34     }
35     // 2 2 2
36     // 2 2 0
37     // 2 0 1
38     return 0;
39 }

```

Dry run

Setup. Take the 3×3 image above, starting at $(1, 1)$ (value 1), $newColor = 2$.

$originalColor = 1 \neq 2$, proceed. Fill(1,1): matches, set to 2; recurse to (0,1): matches, set 2; recurse further to (0,0) and (0,2), both matching, set 2. Back to (1,1)'s other neighbors: (2,1) = 0, no match, skip. (1,0) = 1, matches, set 2; its neighbor (2,0) = 1, matches, set 2. (1,2) = 0, no match, skip. Final grid: top-left 2×2 block plus (0,2) and (2,0) all become 2, while the two 0-pixels and the isolated (2,2) = 1 pixel (not 4-connected to the start through same-colored pixels) remain unchanged.

Complexity

Time Complexity. $O(rows \times cols)$ – each pixel is visited (and re-colored) at most once, since after being set to $newColor$ it no longer matches $originalColor$ and further recursive calls on it return immediately. **Space Complexity.** $O(rows \times cols)$ recursion stack in the worst case (an entirely same-colored grid).

Takeaway

Flood Fill is the simplest possible instance of “traverse an implicit grid graph” – a single source, a single connectivity condition (same original color), and no auxiliary bookkeeping beyond the grid itself (the grid doubles as its own *visited* array, since a re-colored pixel naturally fails the match check on revisit).

1.8 01 Matrix

Problem Statement. Given a binary matrix, return a matrix of the same size where each cell holds the distance to its **nearest** cell containing 0 (distance measured in 4-directional grid steps).

Pattern Recognition

“Distance to the nearest of several targets” is Section 1.6’s multi-source BFS again, applied to a distance-labeling problem instead of a time-to-completion problem: seed the queue with every 0-cell simultaneously (distance 0), then BFS outward, where each level reached corresponds to one more unit of distance.

Algorithm

1. Initialize a *dist* matrix to 0 wherever the input is 0, and push every such cell into the BFS queue.
2. For every other cell, initialize *dist* to -1 (unvisited marker) – this doubles as the *visited* check, exactly as a -1 /sentinel value did in earlier chapters.
3. Run BFS: for each cell popped, examine its 4 neighbors; any neighbor still at -1 gets $dist[neighbor] = dist[current] + 1$ and is pushed.
4. Return the filled *dist* matrix.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<vector<int>> updateMatrix(vector<vector<int>>& mat) {
5      int rows = mat.size(), cols = mat[0].size();
6      vector<vector<int>> dist(rows, vector<int>(cols, -1));
7      queue<pair<int, int>> q;
8
9      for (int r = 0; r < rows; r++) {
10         for (int c = 0; c < cols; c++) {
11             if (mat[r][c] == 0) {
12                 dist[r][c] = 0;
13                 q.push({r, c});
14             }
15         }
16     }
17
18     int dr[] = {-1, 1, 0, 0};
19     int dc[] = {0, 0, -1, 1};
20
21     while (!q.empty()) {
22         auto [r, c] = q.front();
23         q.pop();
24
25         for (int dir = 0; dir < 4; dir++) {
26             int nr = r + dr[dir], nc = c + dc[dir];
27             if (nr >= 0 && nr < rows && nc >= 0 && nc < cols &&
28                 dist[nr][nc] == -1) {
29                 dist[nr][nc] = dist[r][c] + 1;
30                 q.push({nr, nc});

```

```

31     }
32     }
33 }
34 return dist;
35 }
36
37 int main() {
38     vector<vector<int>> mat = {
39         {0, 0, 0},
40         {0, 1, 0},
41         {1, 1, 1}
42     };
43     vector<vector<int>> result = updateMatrix(mat);
44     for (auto& row : result) {
45         for (int x : row) cout << x << " ";
46         cout << endl;
47     }
48     // 0 0 0
49     // 0 1 0
50     // 1 2 1
51     return 0;
52 }

```

Dry run

Setup. Take the 3×3 matrix above.

Initial queue: every 0-cell – $(0, 0)$, $(0, 1)$, $(0, 2)$, $(1, 0)$, $(1, 2)$, all at $dist = 0$. Processing these, their unvisited neighbors get $dist = 1$: $(1, 1)$ (adjacent to $(0, 1)$, $(1, 0)$, and $(1, 2)$, whichever processes first), $(2, 0)$ (adjacent to $(1, 0)$), $(2, 2)$ (adjacent to $(1, 2)$). Next level: $(2, 1)$, adjacent to both $(1, 1)$ (now $dist = 1$) and $(2, 0)/(2, 2)$ (also $dist = 1$), gets $dist = 2$ (it is reached from a $dist = 1$ cell first, since BFS processes level by level). Final matrix matches the expected output, with the single $dist = 2$ cell at $(2, 1)$ being the only cell two steps from the nearest 0.

Complexity

Time Complexity. $O(rows \times cols)$ – each cell is pushed and processed at most once.

Space Complexity. $O(rows \times cols)$ for the $dist$ matrix and the queue.

Takeaway

Rotten Oranges and 01 Matrix are, structurally, the *same* algorithm wearing two different costumes: “minutes until everything is reached” and “distance to the nearest target” are both just **the BFS level number at which each cell is first reached**, read off in two different ways (the level of the very last cell reached, versus the level of each individual cell respectively). Recognizing multi-source BFS by its shape – several simultaneous starting points, uniform step cost – matters more than the specific wording of what the level number represents.

1.9 Number of Islands

Problem Statement. Given a 2D grid of '1' (land) and '0' (water), count the number of **islands** – maximal groups of land cells connected 4-directionally.

Pattern Recognition

This closing problem of the chapter is Section 1.4's connected-component counting loop, applied to an **implicit** grid graph instead of an explicit adjacency matrix – a land cell's neighbors are simply its (up to) 4 adjacent grid cells, with no adjacency list ever constructed.

Algorithm

1. Maintain a *visited* grid (or mutate the input grid directly, marking visited land as '0') and an *islands* counter.
2. For every cell: if it is land and unvisited, increment *islands*, then DFS (or BFS) from it, marking every 4-directionally connected land cell visited.
3. Return *islands*.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void dfs(int r, int c, vector<vector<char>>& grid, vector<vector<bool>>& visited) {
5      int rows = grid.size(), cols = grid[0].size();
6      if (r < 0 || r >= rows || c < 0 || c >= cols) return;
7      if (visited[r][c] || grid[r][c] == '0') return;
8
9      visited[r][c] = true;
10     dfs(r - 1, c, grid, visited);
11     dfs(r + 1, c, grid, visited);
12     dfs(r, c - 1, grid, visited);
13     dfs(r, c + 1, grid, visited);
14 }
15
16 int numIslands(vector<vector<char>>& grid) {
17     int rows = grid.size(), cols = grid[0].size();
18     vector<vector<bool>> visited(rows, vector<bool>(cols, false));
19     int islands = 0;
20
21     for (int r = 0; r < rows; r++) {
22         for (int c = 0; c < cols; c++) {
23             if (grid[r][c] == '1' && !visited[r][c]) {
24                 islands++;
25                 dfs(r, c, grid, visited);
26             }
27         }
28     }
29     return islands;
30 }
31
32 int main() {
33     vector<vector<char>> grid = {
34         {'1', '1', '0', '0'},
35         {'1', '1', '0', '0'},
36         {'0', '0', '1', '0'},
37         {'0', '0', '0', '1'}
38     };
39     cout << numIslands(grid) << endl; // 3
40     return 0;

```

41 }

Dry run

Setup. Take the 4×4 grid above.

(0, 0): land, unvisited, *islands* = 1; DFS visits the top-left 2×2 block of land. (0, 1), (1, 0), (1, 1): already visited, skip. (2, 2): land, unvisited, *islands* = 2; DFS visits just itself (no land neighbors). (3, 3): land, unvisited, *islands* = 3; DFS visits just itself. Final answer: 3 islands.

Complexity

Time Complexity. $O(\text{rows} \times \text{cols})$ – every cell is visited at most once across all DFS calls combined. **Space Complexity.** $O(\text{rows} \times \text{cols})$ for the *visited* grid and, in the worst case, the recursion stack (an entirely land-filled grid).

Takeaway

This chapter's eight problems form a complete toolkit built from just two primitives – DFS and BFS – applied to three settings (an explicit adjacency list, an explicit adjacency matrix, and an implicit grid): plain traversal, traversal with a hash map for building a copy, component-counting via repeated traversal from unvisited starts, and multi-source BFS for simultaneous spreading. Every later chapter in this book's graph coverage layers a new constraint (direction, weight, ordering) on top of exactly these same two traversal primitives.